

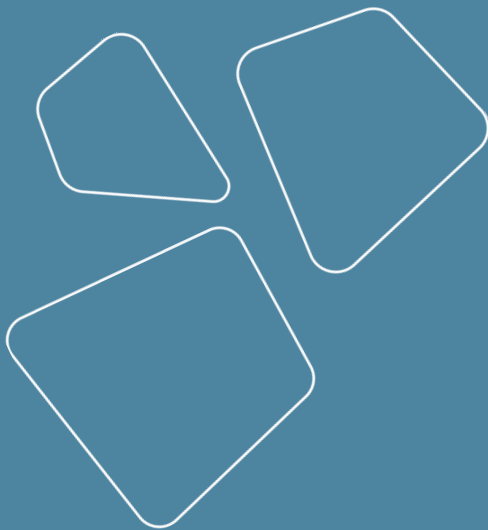
Logging and training frameworks

Vladislav Goncharenko



MIPT, spring 2026

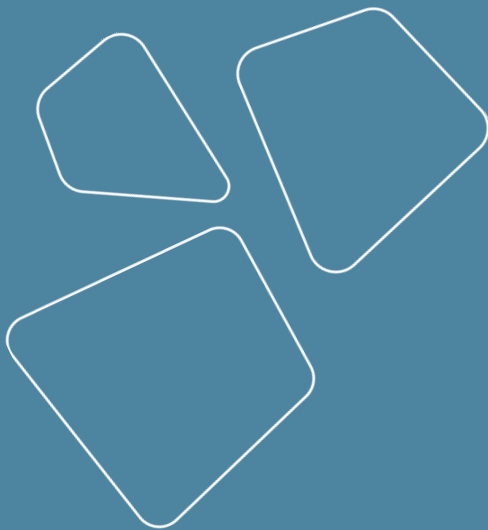
Recap



1. Code management
2. Git servers
3. Workflow
 - a. Distribution
4. CI/CD
5. Codebase structure
 - a. Code formatting
 - b. Inline documentation
6. pre-commit
7. code quality tools

Outline

1. Logging
 - a. MLflow
2. Visualization
3. Frameworks for training
 - a. Lightning
 - b. Hugging face



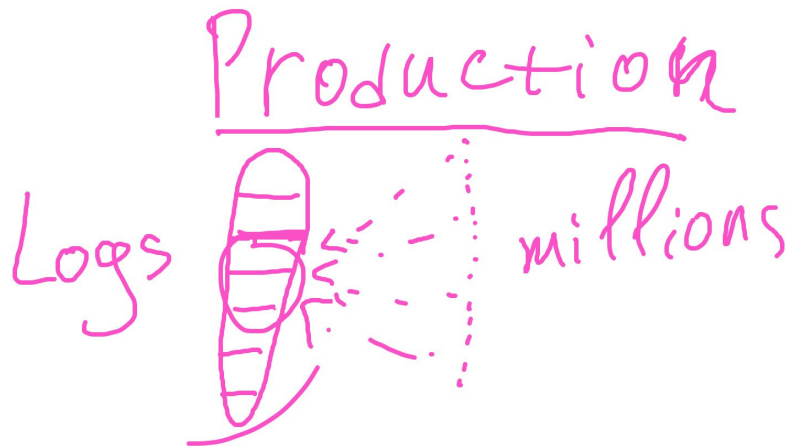
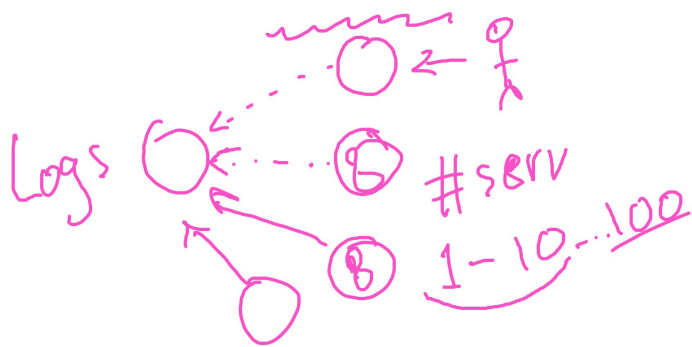
Logging for ML

girafe
ai

01

Requirements for logging in ML

- Structured records
- Comparison of experiments
- Saving artifacts
- Distributed storage
- Realtime writing
- Realtime visualization



Production logging suite

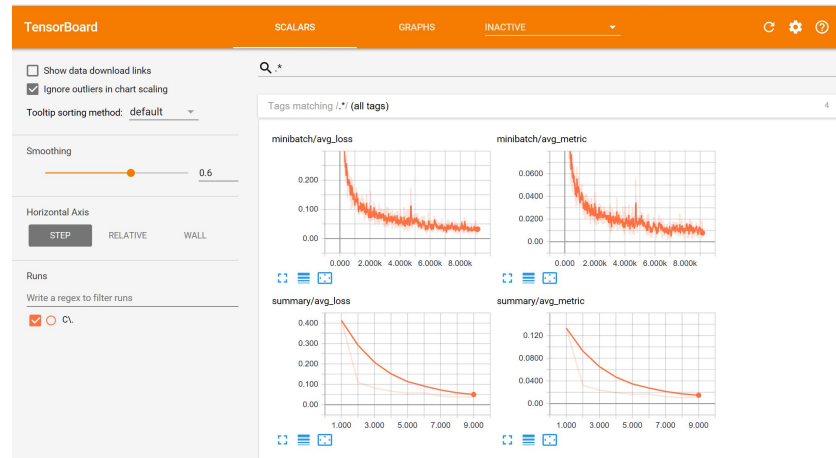
- Prometheus
- Grafana
- Kibana
- ElasticSearch
- GrayLog
- ...

Experiments logging

Becomes more important with more experiments you conduct

Tools here applicable for experiments, but not production monitoring

- absence
- print
- local service
 - tensorboard
- remote service
 - MLflow
 - clear-ml
 - kubeflow
 - w&b, neptune and other proprietary



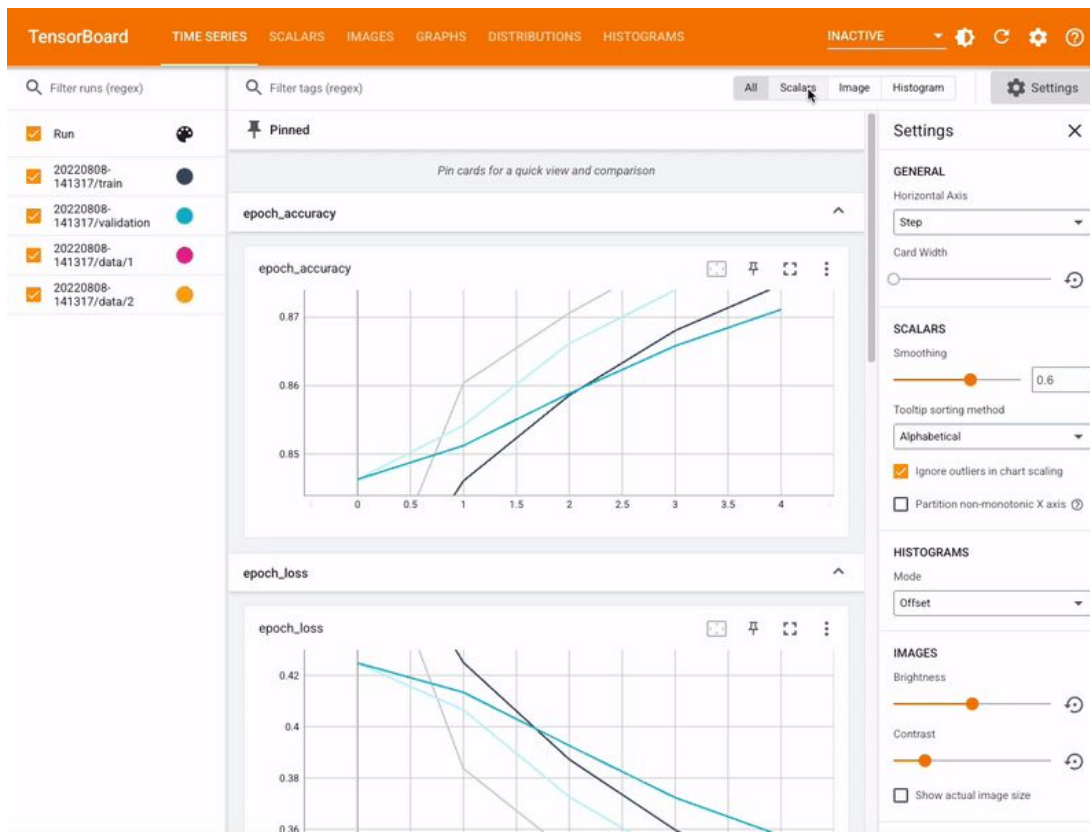
More logging

- [Sacred](#)
- [Flyte](#)
- Comet
- Valohai
- Polyaxon
- Domino Data Lab
- Cortex
-

See overviews: [one](#), [two](#)

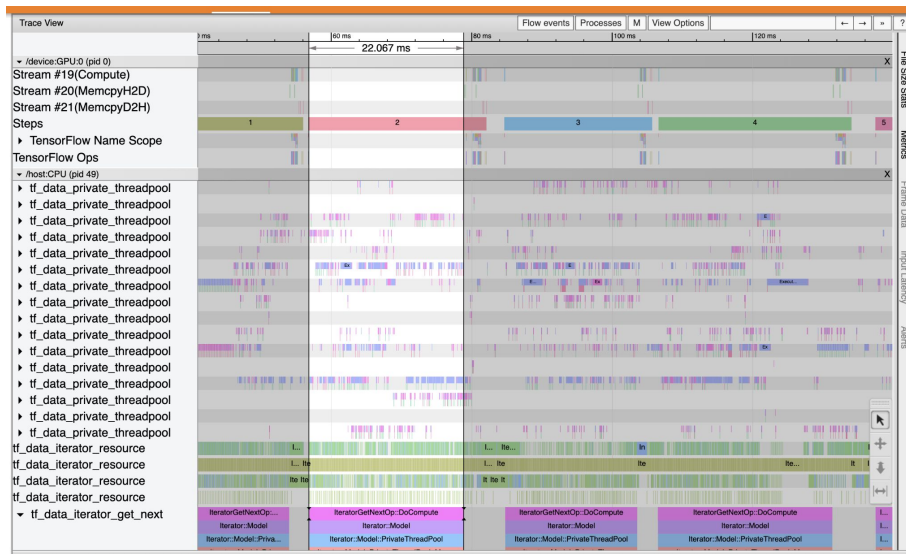
Tensorboard

- Is part of Tensorflow ecosystem but delivered as separate package
- Released in 2017
- FOSS
- Better suited to local running
- Authors wanted to extend it to full fledged solution but didn't succeed (see TensorBoard.dev)
- <https://github.com/tensorflow/tensorboard>
- <https://github.com/lanpa/tensorboardX>
-



Capabilities of Tensorboard

- Tracking and visualizing metrics such as loss and accuracy
- Visualizing the model graph (ops and layers)
- Viewing histograms of weights, biases, or other tensors as they change over time
- Projecting embeddings to a lower dimensional space
- Displaying images, text, and audio data
- Profiling TensorFlow programs



MLflow



- Tracking
 - Record and query experiments: code, data, config, and results
- Projects
 - Package data science code in a format to reproduce runs on any platform
- Models
 - Deploy machine learning models in diverse serving environments
- Model Registry
 - Store, annotate, discover, and manage models in a central repository

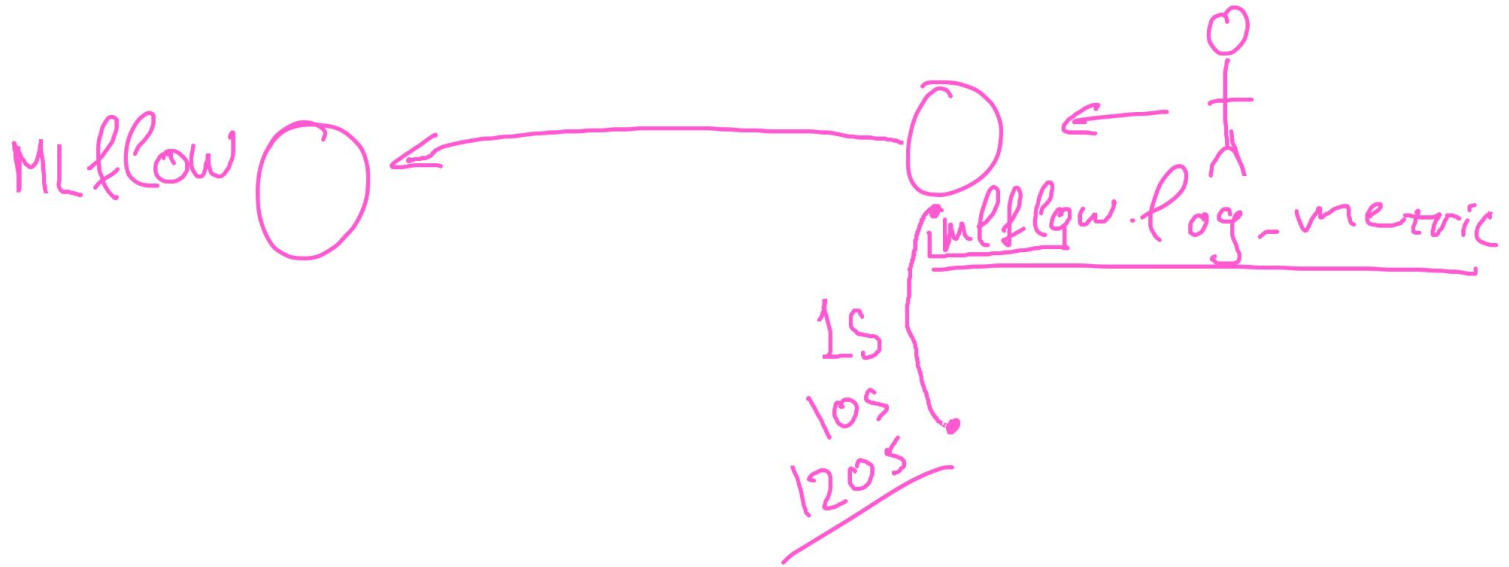
MLflow links

- <https://mlflow.org/docs/latest/getting-started/intro-quickstart/>
- <https://mlflow.org/docs/latest/tracking/autolog>
- https://mlflow.org/docs/latest/deep-learning/pytorch/quickstart/pytorch_quickstart
- https://mlflow.org/docs/latest/api_reference/python_api/mlflow.html#mlflow.set_experiment



Buffer flushing

- can be set to work once in a while



Inference services

Simple deployment tool suitable for relatively small projects.

Alternatives:

- [MLServer](#) (part of [Seldon](#))
- Triton inference server (covered later in this course)

API for inference:

<https://github.com/kserve/open-inference-protocol>



ClearML



<https://clear.ml/>

https://clear.ml/docs/latest/docs/deploying_clearml/clearml_server/

ClearML DataOps

ClearML Hyper-Datasets

ClearML Experiment

ClearML Train

ClearML Reports

ClearML Modelstore

ClearML Pipelines

ClearML Deploy

ClearML Orchestrate

ClearML Schedule

ClearML Compute

ClearML Autoscalers

ClearML Enterprise

Kubeflow

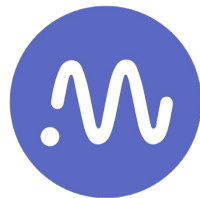
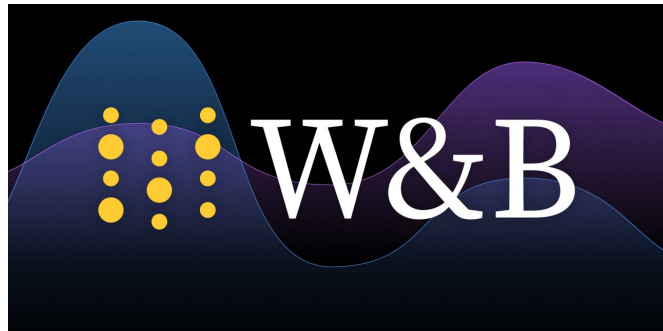
ML suite for Kubernetes (includes tracking)



Kubeflow

W&B, neptune and other proprietary

- neptune.ai
 - good market overviews and knowledge base
- [Weights and biases](https://weightsandbiases.com)
 - provide self-hosted version



neptune.ai

Logging for production

girafe
ai

02

Logging and visualization

- ELK
- Graylog
- Grafana Loki
- Kibana
- Grafana
- Prometheus
-

Visualization

- Kibana
- Grafana
- Prometheus

Visualisation

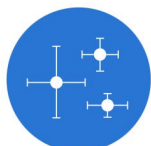
girafe
ai

03

Visualizations

Comprehensive list of visualisations:

<https://datavizcatalogue.com/>



Error Bars



Flow Chart



Flow Map



Gantt Chart



Heatmap



Histogram



Illustration Diagram



Kagi Chart



Line Graph



Marimekko Chart



Multi-set Bar Chart



Network Diagram



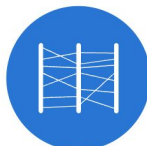
Nightingale Rose Chart



Non-ribbon Chord Diagram



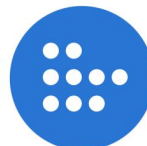
Open-high-low-close Chart



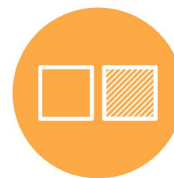
Parallel Coordinates Plot



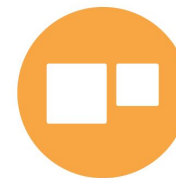
Parallel Sets



Pictogram Chart



Comparisons



Proportions



Relationships



Hierarchy



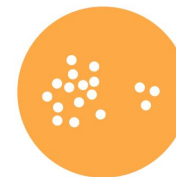
Concepts



Location



Part-to-a-whole



Distribution



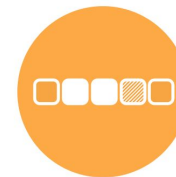
How things work



Processes & methods



Movement or flow



Patterns

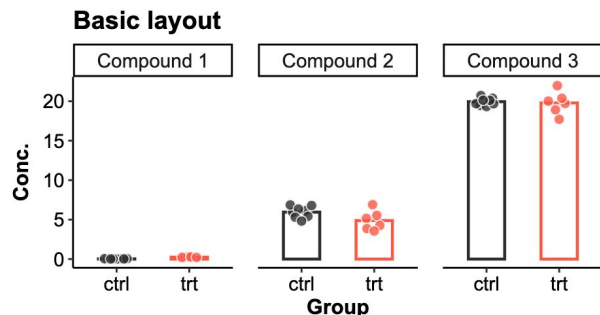
Data visualization initiatives

Comprehensive list of visualisations:

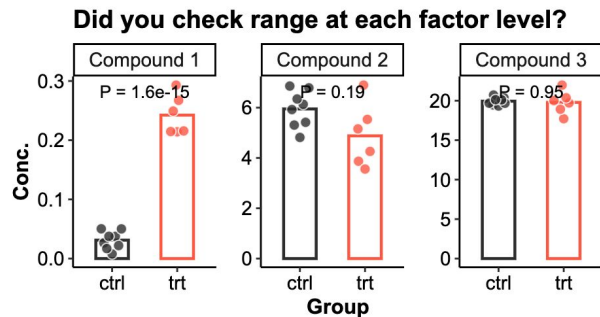
- <https://www.data-to-viz.com/>
 - <https://python-graph-gallery.com/>
 - <https://github.com/holtzy/The-Python-Graph-Gallery>
- <https://datavizcatalogue.com/>

Bad cases of visualization

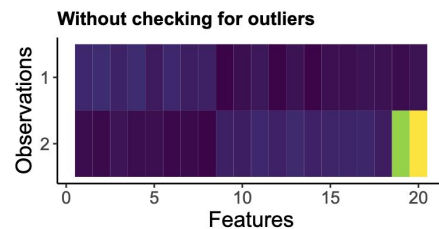
<https://github.com/cxli233/FriendsDontLetFriends>



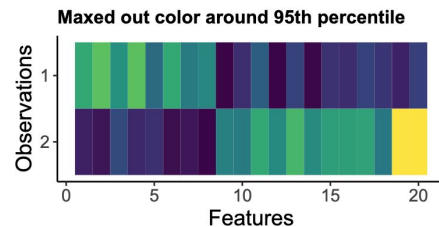
My treatment wasn't doing anything!



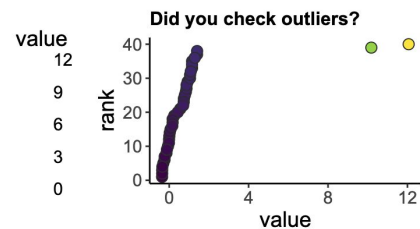
OH!!!



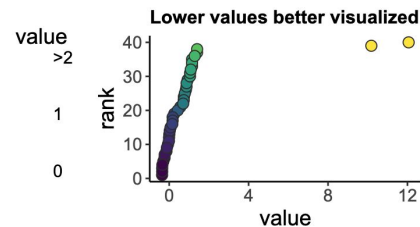
Only two values are different!



Two observations are very different!



Wait...



That's better...

Visualization libraries in Python

- Matplotlib
 - Grandparent of all visualization libs
 - <https://github.com/matplotlib/matplotlib>
- Seaborn
 - <https://github.com/mwaskom/seaborn>
- Plotly <https://github.com/plotly/plotly.py>
 - Proprietary interactive newcomer
 - Dash - web visualizations framework
 - <https://github.com/plotly/dash>
- Bokeh <https://github.com/bokeh/bokeh>
- ggplot (plotnine implementation)
 - <https://github.com/has2k1/plotnine>
 - declarative visualizations with many backends
- Altair <https://github.com/vega/altair>
- Panel <https://github.com/holoviz/panel>

Visualization solutions

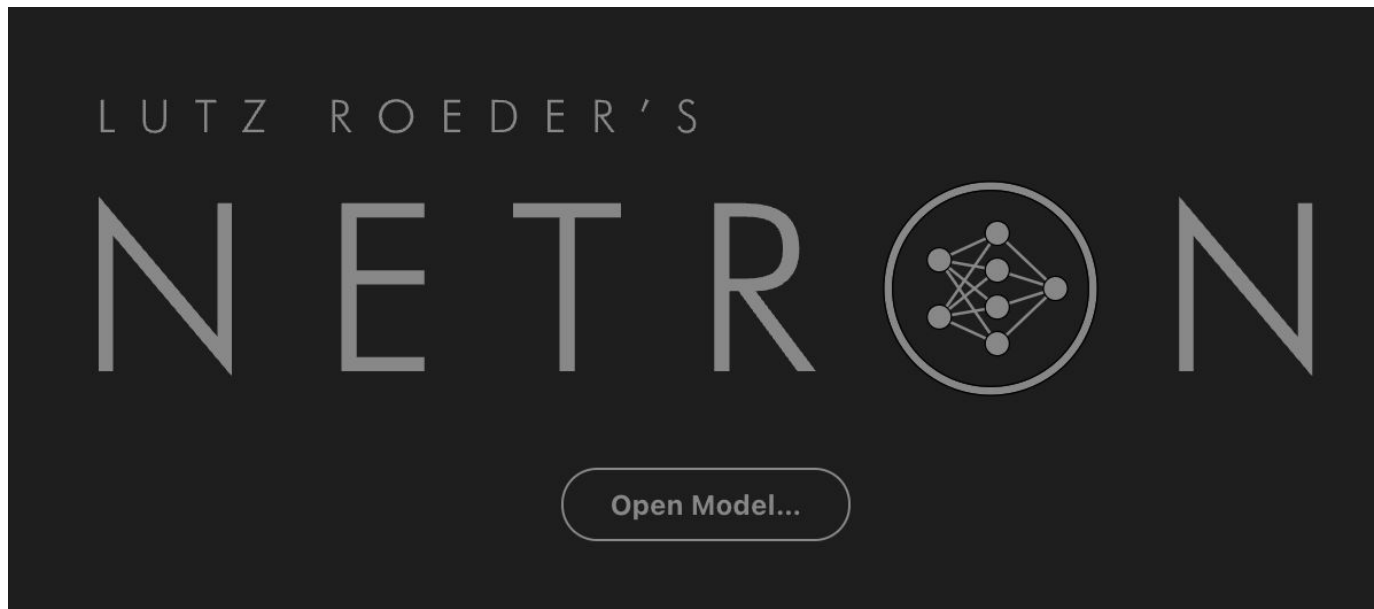
- Superset
- Redash
-

Netron app

Way to visualize your DL models

<https://github.com/lutzroeder/netron>

<https://netron.app/>



Frameworks for training

girafe
ai

04

Experiments Runners

- Pytorch
 - Lightning
 - Hugging face
 - [Ignite](#) (stagnates)
 - [Catalyst](#) (already dead)
 - [great architecture](#)
- [Ray](#)
- Optuna
- DVC Experiments (discarded)
- MLFlow Experiments (discarded)
- [pachyderm](#)
- [fast.ai](#) (very poor coding level e.g. import *)
-



PyTorch Lightning

Catalyst



PyTorch
Ignite

PyTorch Lightning

- Built aiming low entry threshold
 - which affected software design quality
- Currently is the most popular framework for training with PyTorch
- We deployed documentation accessible everywhere
 - <https://lightning.girafe.ai/>
- <https://lightning.ai/docs/pytorch/stable/starter/introduction.html>
- <https://github.com/PyTorchLightning/pytorch-lightning>



PyTorch Lightning

Hugging face (transformers)

- Great for finetuning mainstream tasks
 - Hard to extend beyond default setup
- Recently dropped TF support
- [https://huggingface.co/docs/transformers/main/en/main_classes/trainer#api-reference%20\[%20transformers.Trainer](https://huggingface.co/docs/transformers/main/en/main_classes/trainer#api-reference%20[%20transformers.Trainer)
- https://huggingface.co/docs/transformers/main/en/tasks/question_answering#train
- https://huggingface.co/docs/transformers/v4.45.2/en/tasks/semantic_segmentation#train



Hugging Face

Using particular devices for a process

To avoid interfering with your colleagues (or your own processes) you may want to limit access of a particular run to limited number of GPUs.

It is handful since many torch code uses pure `.cuda()` methods allocating on the first available GPU (don't do that in your code!).

Solution is to setup an environment variable:

- `CUDA_VISIBLE_DEVICES=1 python train.py` - for a single run
- `export CUDA_VISIBLE_DEVICES=0,1` - for the rest of a terminal session

see https://pytorch.org/docs/stable/cuda_environment_variables.html for more environment variables related to cuda.

Profiling in Pytorch

- https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html
- <https://pytorch.org/tutorials/beginner/profiler.html>
- <https://pytorch.org/docs/stable/profiler.html>

<i>Name</i>	<i>Self CPU</i>	<i>CPU total</i>	<i>CPU time avg</i>	<i># of Calls</i>
<i>model_inference</i>	<i>5.509ms</i>	<i>57.503ms</i>	<i>57.503ms</i>	<i>1</i>
<i>aten::conv2d</i>	<i>231.000us</i>	<i>31.931ms</i>	<i>1.597ms</i>	<i>20</i>
<i>aten::convolution</i>	<i>250.000us</i>	<i>31.700ms</i>	<i>1.585ms</i>	<i>20</i>
<i>aten::_convolution</i>	<i>336.000us</i>	<i>31.450ms</i>	<i>1.573ms</i>	<i>20</i>
<i>aten::mkldnn_convolution</i>	<i>30.838ms</i>	<i>31.114ms</i>	<i>1.556ms</i>	<i>20</i>
<i>aten::batch_norm</i>	<i>211.000us</i>	<i>14.693ms</i>	<i>734.650us</i>	<i>20</i>
<i>aten::_batch_norm_impl_index</i>	<i>319.000us</i>	<i>14.482ms</i>	<i>724.100us</i>	<i>20</i>
<i>aten::native_batch_norm</i>	<i>9.229ms</i>	<i>14.109ms</i>	<i>705.450us</i>	<i>20</i>
<i>aten::mean</i>	<i>332.000us</i>	<i>2.631ms</i>	<i>125.286us</i>	<i>21</i>
<i>aten::select</i>	<i>1.668ms</i>	<i>2.292ms</i>	<i>8.988us</i>	<i>255</i>
<i>Self CPU time total: 57.549m</i>				

Dataset splits

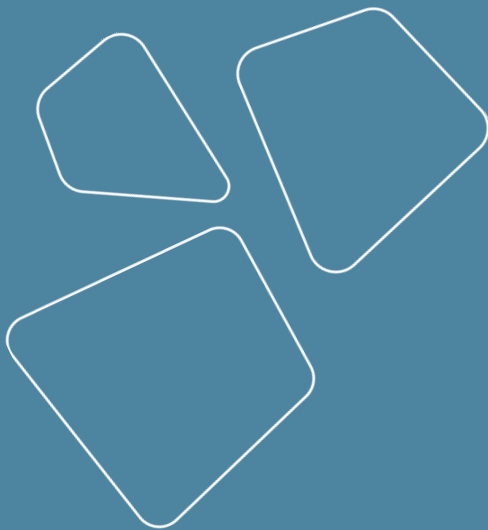
Train-Val-Test scheme is still in power.

But you need to save fixed splits to disk for reproducible results or other method.

Dynamic splits will lead to unreproducible metrics.

Revise

1. Logging
 - a. MLflow
2. Visualization
3. Frameworks for training
 - a. Lightning
 - b. Hugging face



Thanks for attention!

Questions?

